

Speaker Notes

Companion to the presentation deck (10 slides, 8 minutes)

Course: AIN-3009 — MLOps (Term Project)

Author: Mohamed Dribika

Student number: 2280197

Target length: 8 minutes (5-minute cut markers in red)

How to use this document. One slide per page. Each page shows the cumulative timing window, what is on screen, what to say in conversational language, and what to do with your hands or terminal. Practice once with a stopwatch — the bottlenecks tend to be the Hyperparameter Tuning slide and the Drift Monitoring punch line. If running short on time, drop the Hyperparameter Tuning slide first; never cut the Drift Monitoring takeaway.

Slide 1 of 10

Title

0:00 – 0:20 · 20 seconds

ON SCREEN

Cover slide. Bahçeşehir header, project title, your name, student number, course.

WHAT TO SAY

Good morning. My name is Mohamed Dribika, student number 2280197. The project I am presenting is the design and implementation of an end-to-end MLflow-based lifecycle management system, applied to the telco customer churn problem. I have eight minutes to walk through it.

DEMO / CUES

Click into presenter view. Speak slowly on this slide.

Slide 2 of 10

Problem and Dataset

0:20 – 1:00 · 40 seconds

ON SCREEN

Bullets on the left, a small dataset-at-a-glance table on the right with the headline numbers.

WHAT TO SAY

Telecom churn is the canonical use case here. Retention costs roughly five times less than acquisition, so flagging at-risk customers before they cancel is high-value. I picked the IBM Telco Customer Churn dataset — seven thousand customers, twenty features, fifteen categorical and four numeric. Class prevalence is twenty-six point five percent positive, so a mild imbalance. I split it stratified seventy / ten / twenty and held those splits fixed across every experiment.

DEMO / CUES

Point at the dataset-at-a-glance table — let the numbers do the work.

Slide 3 of 10

System Overview

1:00 - 1:40 · 40 seconds

ON SCREEN

Six stage cards in a horizontal flow, with a navy MLflow tracking band beneath them.

WHAT TO SAY

Six MLflow-managed stages. `data_loader` does cleaning and splitting. `train` runs three baselines. `tune` is the Hyperopt search. `registry` handles staging and production promotion. `serve` uses MLflow's built-in scoring server. `monitor` handles drift detection. All six stages point at one MLflow tracking server backed by SQLite and a local artifact root. I keep training and tuning in one experiment, and monitoring in a separate one, so the monitoring time series stays clean.

DEMO / CUES

Sweep your hand left-to-right across the six cards. Tap the navy band to emphasise the single tracking store.

Slide 4 of 10

Experiment Tracking

1:40 - 2:20 · 40 seconds

ON SCREEN

Bullet list of what each run logs; results table with three baselines (LogReg bolded as the strongest).

WHAT TO SAY

First the baselines: Logistic Regression, Random Forest, Gradient Boosting with defaults. Every run logs its parameters, train, validation, and test metrics, an ROC curve, a confusion matrix, and the full scikit-learn Pipeline as a model artifact with an inferred signature. Out of the box, Logistic Regression is the strongest at zero point eight-four ROC-AUC. That is normal for small tabular sets — capacity is not free.

DEMO / CUES

If time allows: alt-tab to the MLflow UI, open the baseline_logreg run, point at the metrics tab — 15 seconds, no more.

5-min cut: skip the UI alt-tab; describe verbally.

Slide 5 of 10

Hyperparameter Tuning

2:20 - 3:10 · 50 seconds

ON SCREEN

Bullets on the left explaining the search; large 0.8457 result card on the right with the winning hyperparameters.

WHAT TO SAY

Then a Hyperopt TPE search — twenty trials over Gradient Boosting. The pattern is parent-and-children: the search itself is one parent MLflow run, and every trial is a nested child. That gives me a sortable trial table in the UI. After tuning, Gradient Boosting reaches zero point eight-four five-seven on test ROC-AUC — now beating Logistic Regression. The winning configuration uses a low learning rate of zero point zero one eight, depth five, heavy leaf regularisation, and subsampling at zero point six three. Conservative.

DEMO / CUES

Point at the big 0.8457 number — that is the headline result.

5-min cut: skip this slide entirely. Mention briefly on Slide 6.

Slide 6 of 10

Model Registry

3:10 - 4:00 · 50 seconds

ON SCREEN

Three lifecycle cards (None → @staging → @production) above a code snippet showing the four CLI commands.

WHAT TO SAY

The best run is promoted to a registered model called telco-churn-classifier. The lifecycle is None, then at-staging, then at-production, using MLflow aliases. Aliases are the MLflow-3 way of doing this, but I also mirror them as a stage tag so the brief's wording — staging and production — is visible in both APIs. The whole lifecycle is four CLI commands: register-best, transition to staging, transition to production, list.

DEMO / CUES

If time: switch to MLflow UI Models tab — show the green production alias on version 2.

Slide 7 of 10

Deployment

4:00 – 5:00 · 60 seconds

ON SCREEN

Two cards. Left: launch command + endpoints. Right: example request payload and the response.

WHAT TO SAY

Deployment uses MLflow's built-in scoring server — zero serving code. The launch command on the left, the standard inference contract on the right. You POST to slash-invocations with a dataframe_split payload — columns and rows. I tested it with a three-row batch from the test split and got back predictions zero, one, zero. The key insight: MLflow rehydrates the full scikit-learn Pipeline, preprocessor and estimator together, from the registry artifact. So the JSON record goes straight to predict with no manual feature engineering in the server. That eliminates the entire training-serving skew bug class.

DEMO / CUES

Have a terminal pre-loaded with the curl command ready to fire. Hit enter, point at the response.

Slide 8 of 10

Drift Monitoring (headline)

5:00 – 6:30 · 90 seconds

ON SCREEN

Big results table on the left (clean / feature_drift / concept_drift); navy insight card on the right with three takeaways.

WHAT TO SAY

This is the headline slide. monitor.py runs three simulated production batches against the production model. Clean — no perturbation. Feature drift — monthly charges shifted up twenty to thirty-five percent, contracts skewed toward month-to-month. Concept drift — features unchanged but twenty-five percent of labels flipped, simulating a market shift. Look at the table. Clean is quiet. Feature drift fires the PSI alert as expected, and interestingly performance stays stable — the model generalises through that shift. But concept drift is the interesting case. PSI is silent because inputs look identical to training, yet ROC-AUC crashes from zero point eight three to zero point six five. That is the lesson. Production monitoring must combine input-distribution checks AND performance metrics on labelled samples. PSI alone is blind to concept drift.

DEMO / CUES

Pause for one full second after the punch line. If a browser tab is open on the Evidently report, show it for ten seconds max.

5-min cut: Do not cut this slide. Cut the Evidently HTML demo if tight.

Slide 9 of 10

What MLflow Gave Us

6:30 – 7:15 · 45 seconds

ON SCREEN

Four numbered cards — single source of truth, registry decouples training from serving, pipeline-as-artifact, one API for the lifecycle.

WHAT TO SAY

Four concrete wins from the project. One: a single source of truth — every parameter, metric, and artifact is in one searchable store, with no ad-hoc spreadsheets. Two: the registry actually decouples training from serving — to redeploy I change an alias, and the serving layer never imports any model code. Three: pipeline-as-artifact — preprocessor and estimator ship together as one model, which eliminates training-serving skew. Four: one client API across the whole lifecycle — small surface area to learn and operate.

DEMO / CUES

Read each headline, briefly paraphrase the body line.

5-min cut: read only the four headlines, skip the bodies.

Slide 10 of 10

Limitations and Q & A

7:15 - 8:00 · 45 seconds

ON SCREEN

Bullets of honest gaps; navy Q-and-A panel at the bottom.

WHAT TO SAY

Honest about what is not done. Threshold is zero point five — production needs cost-aware tuning. There is no automatic rollback on a drift alert — that is the natural next step. The SQLite backend store is fine for a single user; multi-writer production would be Postgres plus object storage. And I did not sweep class-weight strategies because the imbalance is only mild. That said: all five lifecycle objectives are implemented end-to-end and reproducible from the README. Happy to take questions.

DEMO / CUES

End on the navy Q-and-A bar — pause, smile, do not immediately fill silence.

Backup

Likely Questions & Suggested Answers

Q. Why not XGBoost?

A. Sklearn ships Gradient Boosting natively. After tuning, my GB model was within zero point zero zero one ROC-AUC of an XGBoost baseline I tested informally — not worth the extra dependency for this dataset size.

Q. How would you retrain automatically?

A. Schedule train.py on new data, run register-best, then a performance gate decides whether to flip the at-production alias. The MLflow client is the same API throughout.

Q. Is SQLite production-ready?

A. No — it is fine for a single-user demo, file-locked on writes. Postgres for any multi-writer deployment. The artifact root would also move from the local filesystem to S3 or GCS.

Q. Why aliases instead of stages?

A. MLflow 3 deprecates the numeric stages enum in favour of named aliases. Aliases are atomic, reversible, and human-readable. I still write a stage tag alongside them so the brief's wording is visible too.

Q. Why was concept drift invisible to PSI?

A. PSI compares the input distribution against the training reference. In the concept-drift scenario I changed only the labels, not the features. The input histogram is identical, so PSI sees no drift, even though the input/output relationship has shifted.